Name : Sakshi Dattu Bodkhe

Roll No : SYA-14

Class : AIDS

Sub : Python for Data Science

Aim :3. Data Wrangling with pandas: Manipulate and transform data to suit the analysis needs.

Theory:1.What is data wrangling? Difference between data wrangling and data preprocessing

Data Wrangling: Data wrangling is the process of cleaning, organizing, and transforming raw data into a usable format for analysis. It includes tasks like handling missing values, removing duplicates, filtering data, and changing formats.

Data Preprocessing: Data preprocessing is a broader step used mainly in machine learning. It includes: Data wrangling Data normalization Encoding categorical data Feature scaling

Difference: Data Wrangling Data Preprocessing Focuses on cleaning and structuring data Focuses on preparing data for ML models Includes handling missing values, duplicates Includes scaling, encoding, normalization Used in general data analysis Mainly used in machine learning

2. Importance of Exploratory Data Analysis (EDA) in Pandas EDA is used to understand the dataset before analysis.

Importance: Helps to identify missing values Detects outliers and errors Understands data patterns and relationships Helps in better decision making Improves data quality before modeling Example functions in Pandas: df.head() → shows first rows df.describe() → summary statistics df.info() → data types and null values

3. Difference between Series, DataFrame, and Panel Series: One-dimensional data structure Like a single column Example: List of marks

DataFrame: Two-dimensional (rows and columns) Most commonly used in Pandas Like a table or Excel sheet

Panel: Three-dimensional data structure Used for storing multiple DataFrames Note: Panel is now deprecated (removed) in Pandas Difference Table: Feature Series

DataFrame Panel Dimension 1D 2D 3D Structure Single column Table Multiple tables Usage Simple data Main data structure Rare (deprecated)

4. Explain the following functions

a. isnull() Checks for missing values Returns True where data is missing Python code

df.isnull() b. notnull() Opposite of isnull()

Returns True where data is present Python code df.notnull() c. dropna() Removes rows or columns with missing values Python code df.dropna() d. fillna() Fills missing values with a specific value Python code df.fillna(0)

5. What is column transformation and why is it important? Column Transformation: Column transformation means modifying or creating columns to make data more useful. Examples: Changing data type (string ⟶ integer) Creating new columns Converting values (e.g., salary to lakhs) Python code df['Salary_in_Lakhs'] = df['Salary'] / 100000 Importance: Makes data more meaningful Helps in better analysis Improves model performance Converts data into usable format

```
import pandas as pd

# ------------------------------
# Create Sample Data
# ------------------------------
data = {
    'Name': ['Amit', 'Neha', 'Ravi', 'Sita', 'Amit'],
    'Age': [20, 22, 21, 23, 20],
    'Marks': [85, 90, 78, 88, 85],
    'City': ['Pune', 'Mumbai', 'Pune', 'Delhi', 'Pune']
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# ------------------------------
# 🔹 BASIC OPERATIONS
# ------------------------------

# 1. Select single column
print("\nSelect Name column:")
print(df['Name'])

# 2. Add new column
df['Country'] = 'India'

# 3. Sort DataFrame by Marks
df_sorted = df.sort_values(by='Marks')
print("\nSorted Data:")
print(df_sorted)

# 4. Rename column
```

```
df.rename(columns={'Marks': 'Score'}, inplace=True)

# 5. Summary statistics
print("\nSummary Statistics:")
print(df.describe())

# ------------------------------
# 🔹 MODERATE OPERATIONS
# ------------------------------

# 6. Filter rows with multiple conditions
filtered = df[(df['Age'] > 20) & (df['Score'] > 80)]
print("\nFiltered Data:")
print(filtered)

# 7. Group by City and calculate mean Score
grouped = df.groupby('City')['Score'].mean()
print("\nAverage Score by City:")
print(grouped)

# 8. Merge two DataFrames
df2 = pd.DataFrame({
    'Name': ['Amit', 'Neha', 'Ravi', 'Sita'],
    'Salary': [50000, 60000, 45000, 70000]
})

merged = pd.merge(df, df2, on='Name')
print("\nMerged Data:")
print(merged)

# 9. Drop column and row
df_dropped = df.drop(columns=['City'])
df_dropped = df_dropped.drop(index=0)
print("\nAfter Dropping Column and Row:")
print(df_dropped)

# 10. Apply function (increase score by 5)
df['Score'] = df['Score'].apply(lambda x: x + 5)
print("\nUpdated Scores:")
print(df)

# ------------------------------
# 🔹 ADVANCED OPERATIONS
# ------------------------------

# Add Date column for advanced operations
df['Date'] = pd.date_range(start='2024-01-01', periods=len(df))

# 11. Pivot Table
pivot_table = df.pivot(index='Date', columns='Name', values='Score')
print("\nPivot Table:")
print(pivot_table)

# 12. Multi-level Indexing
multi_index = df.set_index(['City', 'Name'])
print("\nMulti-level Index:")
print(multi_index)

# 13. Custom Aggregation Function
def score_range(x):
```

```
    return max(x) - min(x)

agg_result = df.groupby('Name')['Score'].agg(score_range)
print("\nCustom Aggregation (Score Range):")
print(agg_result)

# 14. Time Series Data Wrangling
df.set_index('Date', inplace=True)
monthly_avg = df.resample('M').mean(numeric_only=True)
print("\nMonthly Average:")
print(monthly_avg)
```

```
Original DataFrame:
   Name  Age  Marks    City
0  Amit   20     85    Pune
1  Neha   22     90  Mumbai
2  Ravi   21     78    Pune
3  Sita   23     88   Delhi
4  Amit   20     85    Pune

Select Name column:
0    Amit
1    Neha
2    Ravi
3    Sita
4    Amit
Name: Name, dtype: object

Sorted Data:
   Name  Age  Marks    City Country
2  Ravi   21     78    Pune   India
0  Amit   20     85    Pune   India
4  Amit   20     85    Pune   India
3  Sita   23     88   Delhi   India
1  Neha   22     90  Mumbai   India

Summary Statistics:
             Age      Score
count   5.00000   5.000000
mean   21.20000  85.200000
std     1.30384   4.549725
min    20.00000  78.000000
25%    20.00000  85.000000
50%    21.00000  85.000000
75%    22.00000  88.000000
max    23.00000  90.000000

Filtered Data:
   Name  Age  Score    City Country
1  Neha   22     90  Mumbai   India
3  Sita   23     88   Delhi   India

Average Score by City:
City
Delhi     88.000000
Mumbai    90.000000
Pune      82.666667
```

```
Name: Score, dtype: float64

Merged Data:
   Name  Age  Score    City Country  Salary
0  Amit   20     85    Pune   India   50000
1  Neha   22     90  Mumbai   India   60000
2  Ravi   21     78    Pune   India   45000
3  Sita   23     88   Delhi   India   70000
4  Amit   20     85    Pune   India   50000

After Dropping Column and Row:
   Name  Age  Score Country
```

Conclusion: Data wrangling using Pandas helps in cleaning, transforming, and organizing raw data into a useful format. In this practical, basic operations like selecting, sorting, and summarizing data were performed. Moderate techniques such as filtering, grouping, and merging improved data handling. Advanced methods like pivot tables, multi-level indexing, and time series analysis provided deeper insights. These steps ensure the data is accurate and ready for analysis.